

SAKARYA ÜNİVERSİTESİ
BİLGİSAYAR VE BİLİŞİM BİLİMLERİ
FAKÜLTESİ BİLGİSAYAR
MÜHENDİSLİĞİ BÖLÜMÜ

NESNEYE DAYALI ANALİZ VE TASARIM
DÖNEM SONU PROJE TASARIM RAPORU

Proje Konusu: Akıllı Tedarik ve Lojistik Yönetim Sistemi

Ders Sorumlusu: Sümeyye Kaynak

Tarih: Mayıs 2026

Hazırlayan: Sinan Ulusinan

Öğrenci No: B241210076

Giriş ve Teknolojik Altyapı

Bu rapor, Lojistik Yönetim Sistemi projesinin mimari tasarım kararlarını, kullanılan tasarım desenlerini, neden kullanıldığını ve SOLID prensiplerine uyum sürecini detaylandırmak amacıyla hazırlanmıştır. Ödevde raporu nasıl hazırlayacağımız hakkında bir şablon verilmediği için ödev dosyasında paylaşılan verilerin birleşimi olarak bir rapor oluşturuldu.

Programlama Dili: C#

Platform: .NET (Windows Forms Application)

Geliştirme Ortamı: Visual Studio 2026

Mimari Yapı: Model View Controller (MVC)

Mimari Tasarım: MVC (Model View Controller)

Proje, MVC mimarisine göre katmanlandırılmıştır. Bu tercih, iş mantığı ile arayüzün birbirinden tamamen ayrılmasını sağlayarak sistemin test edilebilirliğini ve bakım kolaylığını artırmıştır. MVC'lerin bu özellikleri sayesinde proje yönetimi de kolaylaşmış olur.

Model: Sistemin verilerini ve iş kurallarını barındırır. Tasarım ve Analiz modelleri bu kısma girer.

View: Kullanıcının etkileşime girdiği Windows Forms ekranlarıdır. İçerisinde iş mantığı barındırmaz, sadece Controller üzerinden gelen veriyi gösterir. Sadece arayüz.

Controller: View'den gelen input ve outputları alır, Model katmanını yönlendirir ve sonucu tekrar View'e iletir. Arayüzde etkileşime girilir.

Kullanılan Tasarım Desenleri Analizi

Sistemdeki karmaşık iş kurallarını polimorfik olarak çözmek ve kodun esnekliğini sağlamak amacıyla aşağıdaki 8 tasarım deseni entegre edilmiştir. Bu desenler de Yaratımsal Yapısal ve Davranışsal olarak üçe ayrılmaktadır.

YARATIMSAL DESENLER

SÜREÇ	TASARIM DESENİ	PROBLEM	NEDEN BU DESEN? SOLİD BAĞLANTISI
Sistem Loglama Logger / AuditService	Singleton	Ödev dosyasında, "Loglama nesnesinden sistem genelinde sadece bir tane olmalıdır" diyor. Birden fazla Logger örneği olursa log dosyaları tutarsızlaşır, aynı anda yazma yarış koşulu yaratır ve kayıtlar kaybolur. Veri kaybımız olur	Sınıfın kendi kurucusunu private yapıp GetInstance() ile tek örneği garanti eder. Thread-safe implementasyon ile eş zamanlı erişimde bile tek Logger nesnesi korunur. ILogger arayüzü ile loglama detayı gizlenir SRP — sadece loglama işi DIP — ILogger arayüzü
Kargo Firması Üretimi Aras / Yurtiçi / GlobalExpres	Factory Method	Sisteme yeni kargo firması (mesela MNG) eklendiğinde nesne üretim kodunun her yerinde if (firma == "MNG") new MngKargo() gibi bloklar ortaya çıkar. Bu açık/kapalı prensibini ihlal eder ve kırılğan koda yol açar. Spagetti code olur.	IKargoServisi arayüzünü implement eden tüm kargo sınıfları bir fabrika üzerinden üretilir. Yeni firma eklemek, yalnızca yeni bir Concrete class + fabrikaya bir kayıt eklemek demektir; mevcut kod değişmez. OCP — kapalı değişime açık genişlemeye DIP — IKargoServisi
Karmaşık Ürün Oluşturma Bilgisayar kasası = RAM + CPU + GPU...	Builder	Şartname, montaj gerektiren ürünlerin (içinde alt bileşenler barındıran birden fazla parça içeren) ayrı yönetilmesini istiyor. Bir new Urun(tip, ad, ram, cpu, gpu, depolama, ekranBoyutu...) konstruktörü çok parametrelili, okunaksız ve hata yapımına açık olur.	UrunBuilder zincir çağrılarla adım adım bir ürün nesnesi inşa eder: .EkleRAM("16GB").EkleCPU("i7").Olustur(). Basit ürünler sade, karmaşık ürünler zengin builder zincirleriyle oluşturulur. Nesne tutarsız durumda hiç yaratılmaz. SRP — üretim mantığı Builder'da LSP — tüm ürünler IUrun'u karşılar

YAPISAL DESENLER

SÜREÇ	TASARIM DESENİ	PROBLEM	NEDEN BU DESEN? SOLİD BAĞLANTISI
Kargo API Entegrasyonu Farklı API yapıları → tek arayüz	Adapter	Aras'ın <code>gonderiOlustur()</code> , Yurtiçi'nin <code>createShipment()</code> , GlobalExpres'in <code>newParcel()</code> metodları farklı parametreler ve dönüş tipleri kullanıyor. Sistem bu API farklılıklarını doğrudan bilmemeli.	Her kargo firması için bir Adapter yazılır (<code>ArasAdapter</code> , <code>YurticiAdapter...</code>). Hepsi ortak <code>IKargoServisi</code> 'ni implement eder. Sistem yalnızca <code>takipNo = servis.GonderiOlustur(paket)</code> der; hangi firmanın API'si olduğu görünmez. DIP — soyut <code>IKargoServisi</code> 'ne bağımlılık OCP — yeni firma = yeni Adapter
Kargo Ücreti Hesaplama Temel + Sigorta + Kırılacak Eşya...	Decorator	Kargo ücreti; ağırlık + mesafe + ek hizmetler (sigorta, kırılacak eşya koruması, ekspres teslimat...) kombinasyonuna göre değişiyor. Tüm kombinasyonlar için ayrı sınıf yazmak ya da iç içe if-else yazmak pratik değil.	Temel <code>KargoUcret</code> nesnesi, <code>Decorator</code> 'larla sarmalanır: <code>new KırılacakDecorator(new SigortaDecorator(new TemelKargo(ağırlık, mesafe)))</code> . Her <code>Decorator</code> üstüne kendi fiyatını ekler. Yeni hizmet = sadece yeni bir <code>Decorator</code> sınıfı. OCP — mevcut kod değişmeden yeni hizmet SRP — her ek hizmet kendi sınıfında

DAVRANIŞSAL DESENLER

SÜREÇ	TASARI M DESENİ	PROBLEM	NEDEN BU DESEN? SOLİD BAĞLANTISI
Sipariş Durum Geçişleri Beklemede → ... → Teslim / İade	State	Her durumda izin verilen işlemler farklı (Kargodaki sipariş iptal edilemez, sadece iade). Bunu if-else ile yönetmek "kargodaysa ve iptal edilmek isteniyorsa ve henüz teslim edilmediyse..." şeklinde içiçe bloklar üretir; şartname bunu yasaklıyor. Bunun yerine farklı bir şey istiyor.	Her sipariş durumu kendi sınıfına taşınır: BeklemedeDurumu, KargodaDurumu, TeslimDurumu... Her sınıf sadece kendi durumunda izin verilen işlemleri uygular. Geçersiz geçiş çağırısı derleme zamanında ya da runtime'da anlamlı exception verir. OCP — yeni durum = yeni sınıf SRP — her durum sınıfı tek sorumlu
Stok Eşiği Bildirimleri Satin Alma (e-posta) + Depo (bildirim)	Observer	Stok düştüğünde iki farklı tarafa (Satin Alma birimine e-posta, Depo sorumlusuna sistem bildirimi) haber gitmeli. Stok sınıfının bu alıcıları doğrudan bilmesi sıkı bağımlılık yaratır ve yeni alıcı eklemek kod değişikliği gerektirir.	ISTokObserver arayüzünü implement eden EmailBildirimObserver ve SistemBildirimObserver nesneleri, Stok sınıfına kayıt olur. Eşik aşıldığında Stok tüm Observer'lara Bildir() çağırır. Yeni alıcı eklemek sadece yeni bir Observer kayıdır. OCP — yeni alıcı = yeni Observer DIP — ISTokObserver arayüzü
Ödeme Yöntemi Seçimi Kredi Kartı / Havale / Kripto...	Strategy	Farklı ödeme yöntemlerinin işlem mantığı tamamen farklı (banka API'si, EFT kanalı, blockchain...). Şartname "ileride Kripto Ödeme gibi yöntemler eklenebilir" diyor; her eklemede mevcut ödeme koduna dokunmak OCP ihlalidir.	IOdemeStratejisi arayüzünü implement eden KrediKartiStratejisi, HavaleStratejisi, KriptoStratejisi sınıfları çalışma zamanında seçilir. Ödeme servisi soyut arayüze bağımlıdır, somut stratejiye değil. OCP — yeni yöntem = yeni Strategy DIP — IOdemeStratejisi arayüzü

SOLID Prensiplerinin Uygulanması

Proje geliştirme sürecinde aşağıdaki SOLID prensipleri temel alınmıştır:

- **S - Single Responsibility:** Logger sınıfı sadece kayıt tutma işinden sorumludur; stok yönetimi veya ödeme işlemleriyle ilgilenmez. Bir sınıf bir iş yapar.
- **O - Open/Closed:** IOdemeStratejisi sayesinde sisteme "Kripto Ödeme" gibi yeni yöntemler, mevcut ödeme kodları değiştirilmeden eklenebilmektedir. Yeni eklenen kodlar eski kodlar değiştirilmeden eklenir.
- **L - Liskov Substitution:** Tüm kargo adapter sınıfları IKargoServisi arayüzünün yerine hatasız bir şekilde geçebilmektedir.
- **I - Interface Segregation:** Arayüzler parçalanarak sınıfların kullanmadıkları metotlara zorlanması engellenmiştir.
- **D - Dependency Inversion:** Video sunumunda kanıtlanacağı üzere, KargoController somut kargo sınıflarına (Aras, Yurtiçi) değil, IKargoServisi soyutlamasına bağımlıdır.